Programming for Problem Solving Using Python
Unit-1

Session-8: Operators- Arithmetic, Assignment, Comparison, Precedence

Iflah Aijaz
School of Engineering & Technology (SOET)
K. R. Managalam University
Program: BTech (CSE)/BTech Full Stack Xebia/BTech UX UI/
B.Sc. DS/B.Sc. Cyber/MC

2025

RECAP

Python Immutable Types: Numbers

- **Integer (`int`):** Whole numbers, positive or negative.

```
x = 42
y = -7

```

- **Float (`float`):** Decimal numbers.

```
pi = 3.14159
e = 2.71828

```

- **Complex (`complex`):** Numbers with real and imaginary parts.

RECAP

```
z = 1 + 2j

```

Characteristics:

- Immutable: Cannot be changed after creation.
- Common operations: Addition, subtraction, multiplication, division, etc.

```
sum = 7 + 3
difference = 10 - 4
product = 2 * 3
quotient = 15 / 5

```

RECAP

```
full_greeting = greeting + " " + name   # 'Hello, World! Alice'
first_letter = name[0]   # 'A'
substring = greeting[0:5]   # 'Hello'
length = len(name)   # 5

```

RECAP

Tuples:

- Ordered collection of items.
- Enclosed in parentheses.
- Can contain items of different data types.

```
coordinates = (10, 20)
person = ("Alice", 30, "New York")

```

Characteristics:

- Immutable: Cannot be changed after creation.

RECAP

- Common operations: Indexing, slicing, concatenation.

```
first_item = coordinates[0]   # 10
sub_tuple = person[0:2]   # ('Alice', 30)
combined = coordinates + (30, 40)   # (10, 20,
    30, 40)
length = len(person)   # 3

```

Session-8

Operators- Arithmetic, Assignment, Comparison

Arithmetic Operators

Python supports several arithmetic operators for performing mathematical operations. Here are the main ones:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Integer Division (//)
- Modulus (%)
- Exponentiation (**)

Arithmetic Operators (Examples)

Let's see some examples of using arithmetic operators in Python:

- Addition:

  3 + 5 = 8

- Subtraction:

  7 - 2 = 5

- Multiplication:

  4 * 6 = 24

Arithmetic Operators (More Examples)

Continuing with examples:

- Division:

```
10 / 3 = 3.3333333333333335
```

- Integer Division:

```
10 // 3 = 3
```

- Modulus:

```
10 % 3 = 1
```

- Exponentiation:

Arithmetic Operators Table

There are 7 arithmetic operators in Python. The lists are given below:

	Operator	Description	Syntax
Addition Operator	+	Addition: adds two operands	x + y
Subtraction Operator	–	Subtraction: subtracts two operands	x – y
Multiplication Operator	*	Multiplication: multiplies two operands	x * y
Division Operator	/	Division (float): divides the first operand by the second	x / y
Floor Division Operator	//	Division (floor): divides the first operand by the second	x // y
Modulus Operator	%	Modulus: returns the remainder when the first operand is divided by the second	x % y
Exponentiation Operator	**	Power (Exponent): Returns first raised to power second	x ** y

Arithmetic Operators Precedence

Precedence of Arithmetic Operators in Python

Let us see the precedence and associativity of Python Arithmetic operators.

Operator	Description	Associativity
**	Exponentiation Operator	right-to-left
%, *, /, //	Modulos, Multiplication, Division, and Floor Division	left-to-right
+, −	Addition and Subtraction operators	left-to-right

Python Program- Arithmetic Operators

```python
# Python program demonstrating arithmetic operators

a = 10
b = 5

# Addition
print("Addition:", a + b)

# Subtraction
print("Subtraction:", a - b)
```

Python Program- Arithmetic Operators (contd.)

```
# Multiplication
print("Multiplication:", a * b)

# Division
print("Division:", a / b)
```

Python Program- Arithmetic Operators (contd..)

```
OUTPUT : -
  # Integer Division
   print("Integer Division:", a // b)

   # Modulus
   print("Modulus:", a % b)

   # Exponentiation
   print("Exponentiation:", a ** b)

```

```
   The output of the program will be:
   Addition: 15
   Subtraction: 5
   Multiplication: 50
   Division: 2.0
   Integer Division: 2
```

figures/KRMU

Python Assignment Operators

Assignment Operators:
Assignment operators are used to assign values to variables.
Syntax:-

- Simple assignment: =
- Add and assign: +=
- Subtract and assign: -=
- Multiply and assign: *=
- Divide and assign: /=
- Modulus and assign: %=

Python Assignment Operators

- Floor divide and assign: //=
- Exponent and assign: **=
- Bitwise AND and assign: &=
- Bitwise OR and assign: |=
- Bitwise XOR and assign: ^
- Bitwise right shift and assign: >>=
- Bitwise left shift and assign: <<=

Python Assignment Operators

Syntax:-

```
x = 10          # = operator
x += 5          # x = x + 5
x -= 3          # x = x - 3
x *= 2          # x = x * 2
x /= 4          # x = x / 4
x %= 3          # x = x % 3
x //= 2         # x = x // 2
x **= 3         # x = x ** 3
x &= 6          # x = x & 6
x |= 2          # x = x | 2
x ^= 4          # x = x ^ 4
x >>= 1         # x = x >> 1
x <<= 2         # x = x << 2
```

Python Program Demonstrating Assignment Operators

```python
# Python program demonstrating assignment operators

# Initial values
a = 10
b = 5

print("Initial values: a =", a, ", b =", b)

# Using assignment operators
a += b  # a = a + b
print("After a += b, a =", a)  # a = 15

a -= b  # a = a - b
print("After a -= b, a =", a)  # a = 10
```

figures/KRIWI

Python Program Demonstrating Assignment Operators

```
a *= b  # a = a * b
print("After a *= b, a =", a)  # a = 50

a /= b  # a = a / b
print("After a /= b, a =", a)  # a = 10.0

a %= b  # a = a % b
print("After a %= b, a =", a)  # a = 0.0

a **= b  # a = a ** b
print("After a **= b, a =", a)  # a = 0.0

a //= b  # a = a // b
```

figures/KRMU

Python Program Demonstrating Assignment Operators

```
print("After a //= b, a =", a)   # a = 0.0

# Bitwise assignment operators
a = 10   # Resetting a to 10
a &= b   # a = a & b
print("After a &= b, a =", a)   # a = 0

a |= b   # a = a | b
print("After a |= b, a =", a)   # a = 5

a ^= b   # a = a ^ b
print("After a ^= b, a =", a)   # a = 0

a >>= 1   # a = a >> 1
```

Python Program Demonstrating Assignment Operators

```
print("After a >>= 1, a =", a)   # a = 0

a <<= 1  # a = a << 1
print("After a <<= 1, a =", a)   # a = 0

Output of the Program:-

Initial values: a = 10, b = 5
After a += b, a = 15
After a -= b, a = 10
After a *= b, a = 50
After a /= b, a = 10.0
After a %= b, a = 0.0
After a **= b, a = 0.0
```

Python Program Demonstrating Assignment Operators

```
After a //= b, a = 0.0
After a &= b, a = 0
After a |= b, a = 5
After a ^= b, a = 0
After a >>= 1, a = 0
After a <<= 1, a = 0
```

Python Comparison Operators

Comparison Operators:

- Comparison operators are used to compare two values.
- Examples:
  - < (Less than)
  - > (Greater than)
  - <= (Less than or equal to)
  - >= (Greater than or equal to)
  - == (Equal to)
  - ! = (Not equal to)

Python Program Demonstrating Comparison Operators

Example:-

```
a = 10
b = 20

print(a == b)   # False
print(a != b)   # True
print(a > b)    # False
print(a < b)    # True
print(a >= b)   # False
print(a <= b)   # True
```

Test Your Knowledge- Fill in the Blanks

1. Arithmetic operators perform ____operations on numbers.
2. The assignment operator in Python is represented by the symbol ____.
3. Comparison operators are used to compare two values and return a _____value.
4. The equality operator in Python is represented by the symbol ____.
5. The modulo operator returns the _____of the division operation.

Test Your Knowledge- Fill in the Blanks (Solutions)

1. Arithmetic operators perform mathematical operations on numbers.
2. The assignment operator in Python is represented by the symbol =.
3. Comparison operators are used to compare two values and return a Boolean value.
4. The equality operator in Python is represented by the symbol ==.
5. The modulo operator returns the remainder of the division operation.

Test Your Knowledge- MCQ

1. What is the result of the following expression in Python?

   `10 / 3`

   a. 3
   b. 3.33
   c. 3.0
   d. 3.3333333333333335

2. What is the result of the following expression in Python? `7 // 2`

   a. 3
   b. 3.5
   c. 3.0
   d. 3.3333333333333335

Test Your Knowledge- MCQ

3. Which operator is used to calculate the remainder of a division?

   a. %
   b. //
   c. /
   d. **

4. What is the result of the expression 4 < 5 and 5 < 6?

   a. True
   b. False
   c. Error
   d. None of the above

Test Your Knowledge- MCQ

5. What will be the output of the following code?
   print(3 ** 3)

   a. 9
   b. 27
   c. 81
   d. 6

6. Which operator is used to perform floor division?

   a. %
   b. //
   c. /
   d. **

Test Your Knowledge- MCQ

7. What will be the output of the following code?
```
print(9 % 4)
```

   a. 1
   b. 2
   c. 3
   d. 0

8. What is the result of the following expression in Python? 10 * (3 + 5) // 2

   a. 40
   b. 35
   c. 20
   d. 25

Test Your Knowledge- MCQ (Solutions)

1. d. 3.3333333333333335
2. a. 3
3. a. %
4. a. True
5. b. 27
6. b. //
7. a. 1
8. a. 40

Test Your Knowledge- Interview Questions

1. What are arithmetic operators in Python?
2. Explain the difference between the '/' and '//' operators in Python.
3. What is the modulus operator used for in Python?
4. How can you perform exponentiation in Python?
5. Explain the order of precedence for arithmetic operators in Python.
6. What happens if you try to perform arithmetic operations between different data types in Python?

Test Your Knowledge- Interview Questions

7. How do you handle division by zero in Python?
8. Can you use arithmetic operators with complex numbers in Python? If yes, how?
9. Discuss the use of parentheses in arithmetic expressions and how they affect the order of evaluation.
10. Explain the concept of augmented assignment operators in Python and provide examples.

Test Your Knowledge- Interview Questions (Solutions)

1. Ans:- Arithmetic operators in Python are used to perform mathematical operations on numbers. These include addition (+), subtraction (-), multiplication (∗), division (/), floor division (//), modulus (%), and exponentiation (∗∗).
2. Ans:- The '/' operator performs regular division and returns a floating-point result. The '//' operator performs floor division and returns the nearest integer value less than or equal to the quotient.

Test Your Knowledge- Interview Questions (Solutions)

3. Ans:- The modulus operator (%) returns the remainder of the division operation between two numbers.
4. Ans:- Exponentiation in Python is performed using the ** operator. For example, $2 * *3$ calculates 2^3, resulting in 8.
5. Ans:- Parentheses, Exponentiation, Multiplication, division, floor division, and modulus (same precedence), Addition and subtraction (evaluated left to right).

Test Your Knowledge- Interview Questions (Solutions)

6. Ans:- Python will automatically convert the operands to a common data type before performing the operation. For example, if you try to add an integer and a float, Python will convert the integer to a float before performing the addition.
7. Ans:- Division by zero in Python raises a ZeroDivisionError exception. You can handle this exception using a try-except block to prevent your program from crashing.

Test Your Knowledge- Interview Questions (Solutions)

8. Ans:- Yes, you can use arithmetic operators with complex numbers in Python. The operators work as expected, performing operations on the real and imaginary parts separately.
9. Ans:- Parentheses are used to override the default order of precedence in arithmetic expressions. Expressions within parentheses are evaluated first, followed by the rest of the expression according to the standard order of precedence.

Test Your Knowledge- Interview Questions (Solutions)

10. Ans:- Augmented assignment operators in Python are shortcuts for performing arithmetic operations and assigning the result back to the variable. Eg. $x + = 5$ is equivalent to $x = x + 5$, $y - = 3$ is equivalent to $y = y - 3$ and $z * = 2$ is equivalent to $z = z * 2$.

Thank You!

Any Questions?